NestJS Monorepo: E-Commerce Flash Sale Engine

Real-World Microservices: Orders, Inventory Caching, Redis Pub/Sub, BullMQ & Resend Email

Business Context: Imagine an online storefront (like Amazon or Shopify) hosting a high-traffic "Flash Sale." Thousands of customers place orders simultaneously. If the API spends time synchronously writing to MySQL and sending order receipt emails on every request, the server will crash. To solve this, the student will build a **NestJS Monorepo** that uses **Redis Caching** for speed, **Redis Pub/Sub** for real-time order events, **BullMQ queues** for background email processing, and a **Cron Job** for daily abandoned cart reminders.

1. Monorepo Architecture Blueprint

```
flash-sale-monorepo/
├── apps/
│   ├── store-api/            # Order Creation REST API & Customer Management
│   └── fulfillment-service/  # Redis Pub/Sub Subscriber, BullMQ Queues, Cron & Email
├── libs/
│   └── common/               # Shared MySQL Schemas (Prisma/TypeORM), Redis Setup, Email DTOs
├── nest-cli.json
└── package.json
```

2. Tech Stack & Service Boundaries

App 1: `store-api`

- **Interface:** REST API Gateway
- **Primary Focus:** Fast Order Creation, Customer Signups
- **Speed Layer:** Redis Inventory Cache (atomic decrement)
- **Transport:** Redis Pub/Sub Publisher

App 2: `fulfillment-service`

- **Interface:** Microservice / Background Worker
- **Responsibilities:** Order Processor, Scheduled Cron Jobs, BullMQ Queue
- **Integrations:** Resend API / SendGrid (Transactional Email)

3. Functional Requirements & Detailed Descriptions

A. Store API (`apps/store-api`)

`POST /api/v1/orders`

WHY THIS REQUIREMENT EXISTS

During flash sales, hitting MySQL directly to check stock for thousands of concurrent users causes database locking and crashes. Stock checks and decrements must happen in Redis memory (under 10ms).

- **Inventory Check:** Reads product stock from Redis (e.g., `SET item:iphone15:stock 100`).
- **Atomic Decrement:** Uses Redis `DECR` to decrement stock. If stock reaches below 0, immediately returns `400 Bad Request ("Out of Stock")` without touching MySQL.
- **Event Broadcast:** If stock is available, publishes an `order.created` event over Redis Pub/Sub containing order details and returns `202 Accepted ("Order Placed")`.

```
// Request Payload: POST /api/v1/orders
{
  "customerId": "cust_1029",
  "customerEmail": "customer@gmail.com",
  "productId": "iphone15",
  "quantity": 1,
  "totalPrice": 999.00
}
```

POST /api/v1/cart/item

> **WHY THIS REQUIREMENT EXISTS**
>
> Allows customers to add items to their shopping cart. Tracks cart activity in MySQL so our Cron job can identify abandoned carts later.
>
> - Saves a cart record in MySQL with `status = 'ACTIVE'` and records a timestamp `lastUpdatedAt`.

B. Fulfillment & Notification Engine (`apps/fulfillment-service`)

1. Real-Time Order Event Subscriber (Redis Pub/Sub)

WHY THIS REQUIREMENT EXISTS

The API shouldn't make the customer wait while an email sends or heavy fulfillment logic runs. The Fulfillment Service listens to order events asynchronously and delegates tasks to background queues.

- Subscribes to the `order.created` channel over Redis Pub/Sub.
- Performs permanent DB record creation in MySQL (`orders` table).
- Pushes an `OrderReceiptJob` payload into a BullMQ queue called `notifications-queue`.

2. Background Email Worker (BullMQ Queue + Resend API)

WHY THIS REQUIREMENT EXISTS

Third-party email APIs (like Resend or SendGrid) can experience network blips or rate limits. Using a BullMQ queue guarantees that no order receipt email is ever lost, using retries if the email provider fails.

- Listens to jobs on `notifications-queue`.
- Integrates with a free transactional email SDK (**Resend API** or **Nodemailer**).
- Renders a formatted HTML email containing the Order ID, Item details, and Price.
- **Resiliency Rule:** Configured with **3 automatic retries and exponential backoff** (retry after 5s, 15s, 45s) if the email API throws a 5xx error.

```
// Sample HTML Email Sent to Customer
┌──────────────────────────────────────────────────────┐
│ Subject: Order Confirmation #ORD-98214 - Flash Sale Store│
├──────────────────────────────────────────────────────┤
│ Hello Customer,                                      │
│ Thank you for your purchase! Here is your order receipt:  │
│                                                      │
│  - Product: iPhone 15                                │
│  - Quantity: 1                                       │
│  - Total Paid: $999.00                               │
│                                                      │
│ We are preparing your order for shipment!            │
└──────────────────────────────────────────────────────┘
```

3. Scheduled Cron Job: Abandoned Cart Notification Engine

WHY THIS REQUIREMENT EXISTS

E-commerce businesses recover millions of dollars by emailing users who added items to their cart but never completed checkout. A Cron job automates this daily business workflow.

- Implement a scheduled task using NestJS `@Cron(CronExpression.EVERY_DAY_AT_MIDNIGHT)` (or set to every 1 minute for testing/demoing).
- **Database Query:** Queries MySQL for all shopping carts where `status == 'ACTIVE'` and `lastUpdatedAt` is older than 24 hours.
- **Queue Dispatch:** For each abandoned cart found, pushes an `AbandonedCartJob` into `notifications-queue`.
- **Email Trigger:** The worker sends a friendly email: *"You left something behind! Complete your order now and get 10% off."*

4. Verification & Demo Checklist

- **Monorepo Startup Check:** Show starting both services independently using `nest start store-api` and `nest start fulfillment-service`.
- **Flash Sale Out-of-Stock Test:** Seed Redis stock to `5`. Fire 10 fast order requests. Prove that the first 5 succeed with `202 Accepted`, and the remaining 5 instantly fail with `400 Bad Request ("Out of Stock")`.
- **End-to-End Email Order Receipt Test:** Place a valid order. Show that the API Gateway responds immediately, the Pub/Sub subscriber catches the event, BullMQ processes the queue item, and an actual confirmation email arrives in your inbox.
- **Abandoned Cart Cron Test:** Insert an inactive cart in MySQL. Trigger the Cron job manually or via schedule, and prove that an automated discount email is queued and delivered.